Formal Verification of a One-Master One-Slave SystemC/TLM AMBA AHB Model with UPPAAL Timed Automata

C. Deepthi Sree (BL.EN.U4ECE23107), G. Sohan (BL.EN.U4ECE23117), K. Lochana Reddy (BL.EN.U4ECE23127)

Department of Electronics and Communication Engineering, Amrita Vishwa Vidyapeetham
23ECE337 System on Chip — Simulation and Analysis-based Assignment, Batch B, Semester VII

***Abstract*—We repeat, for the one-master one-slave (1M1S) configuration, the model-checking experiment of Pockrandt, Herber and Glesner on a SystemC/TLM 2.0 model of the AMBA AHB bus [1]. Starting from the UPPAAL timed-automata model of the design, we check the three properties of the paper's Table II that apply to 1M1S: deadlock freedom, eventual bus grant to master 1, and a bound on communication time after a grant. For the timing property we add one clock to the model, as the paper describes, and search for the smallest bound that holds. We also recreate the missing counter reset reported in the paper and check how it shows up in our model. Every number in Section IV is produced by a script from saved UPPAAL output.**

***Index Terms*—AMBA AHB, SystemC, transaction-level modeling, timed automata, UPPAAL, model checking.**

I. Introduction

Transaction-level models (TLM) in SystemC let designers simulate a system on chip long before RTL exists, but a simulation run follows only one ordering of concurrent processes. Corner cases in bus protocols, such as a counter that is not reset after a split transaction, can therefore survive many simulation runs. Model checking explores every reachable state of a formal model instead.

Reference [1] transforms SystemC/TLM designs into UPPAAL timed automata [3], [4] and applies the method to an industrial TLM 2.0 model of the AMBA AHB [5]. Our assignment repeats that experiment for the smallest configuration, one master and one slave (1M1S), and compares our measurements with the 1M1S column of the paper's Table II.

II. Reference Work

The key contribution of [1] is an optimised encoding of the TLM 2.0 payload event queue (PEQ). Instead of a global clock array and a sorted queue, each pending notification is held by its own automaton with a local clock `c` (paper Fig. 4), and a second automaton invokes the callback (paper Fig. 5). This reduces the number of clocks and makes symmetry easier to exploit.

The AHB case study has a `tlm2_master` that issues read and write bursts by blocking transport to a `master_to_ahb` adapter, which splits them into AHB transfers over the bus to `ahb_mem` slaves. The paper checks four properties for 1M1S up to 2M2S. For 1M1S it reports 74,345 states, with 0:07 for deadlock freedom, 0:05 for bus grant to M1 and 0:17 for communication in time. It also reports a bug found during verification: a counter that stores the number of completed transfers before a transaction is split was not reset, so all split transactions after the first one failed (Section VI-B of [1]).

III. Model and Method

A. The 1M1S model

Our model `amba_AHB_1M1S.uppaal.xml` contains 66 automata instances built from 44 templates: the SystemC scheduler and initialiser, the clock `clk` (period 20 time units), the traffic generator `TLM2_M1`, the adapter `AHB_M1`, the bus `AHB_Bus` and the memory `AHB_S1`. We could not regenerate it with the STATE tool, so it was reduced by hand from the 2M2S model: the master-2, slave-2 and second traffic-generator instances were removed, `number_of_slaves` in the traffic generator was changed from 2 to 1, and the initial `readyprocs` count from 3 to 2. The bus templates are unchanged and still loop over two masters and two slaves; the unused master-2 and slave-2 ports are bound to channels that no automaton uses. In each round the traffic generator writes and then reads a 3-beat burst at address 0, waits (`wait_clks = 20`) and starts again, so the model runs forever.

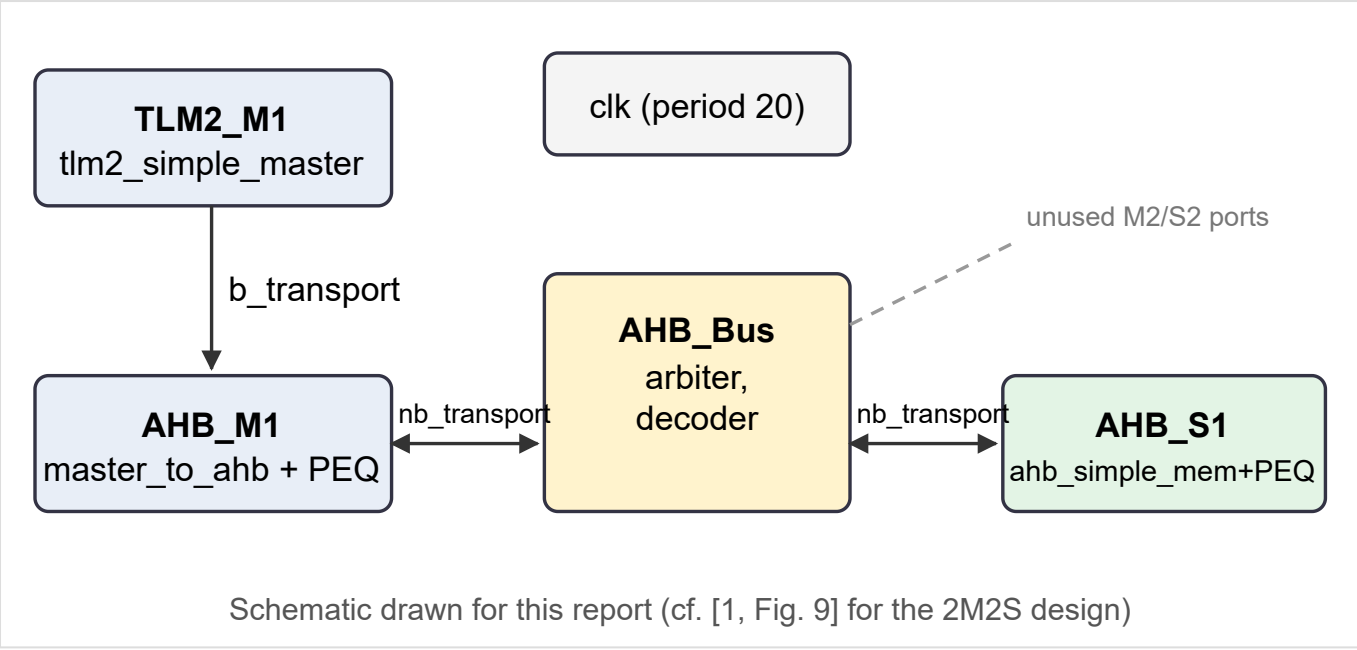


Fig. 1. 1M1S configuration checked in this work.

B. PEQ encoding in our model

Each PEQ is built from three templates: `peq_notify` accepts a notification and hands it to a free element, four `peq_element` instances each hold one notification with a local clock `c` and the invariant `c <= delay` (the automaton of paper Fig. 4, Fig. 2 here), and `peq_fetch` calls the callback and signals `deactivate` (paper Fig. 5, Fig. 3 here). The model has no global PEQ clock array, so it uses the optimised encoding of [1].

C. Properties

Deadlock freedom is checked as `A[] not deadlock`. For bus grant we use the leads-to property `AHB_M1$m_bus_req_pending --> AHB_M1$m_bus_granted`: whenever the adapter has a pending bus request it is eventually granted. A reachability query `E<>` would only show that one such run exists, so we report it separately. The properties that involve master 2 do not apply to 1M1S.

D. Timing instrumentation

For communication in time, [1] adds an extra clock `x` that is reset whenever a master is granted the bus. We generate `amba_AHB_1M1S_timed.uppaal.xml` from the unchanged model with a script that adds a global clock `x` and a flag `M1_isCommunicating`. Both edges where the adapter records a grant set `x = 0` and the flag to true; both edges where `b_transport` finishes a transaction set the flag to false. The new variables are never read by the model, so its behaviour is unchanged, although the extra clock enlarges the state space. We then search for the smallest `N` for which `A[] M1_isCommunicating imply x <= N` holds, by doubling from one clock period and then bisecting.

E. Bug study

The adapter resets `m_txn_count = 0` when a write or read transaction is cleaned up (`tlm2_master_to_ahb.cpp` lines 120 and 154 of the STATE-2.1 `amba_AHB` example, included in `systemc_source/`; the UPPAAL model merges each clean-up into one update annotated with lines 116 and 150). A second derived model removes both resets. We check deadlock freedom and the liveness property `M1_isCommunicating --> !M1_isCommunicating` (every started transfer finishes) on both versions.

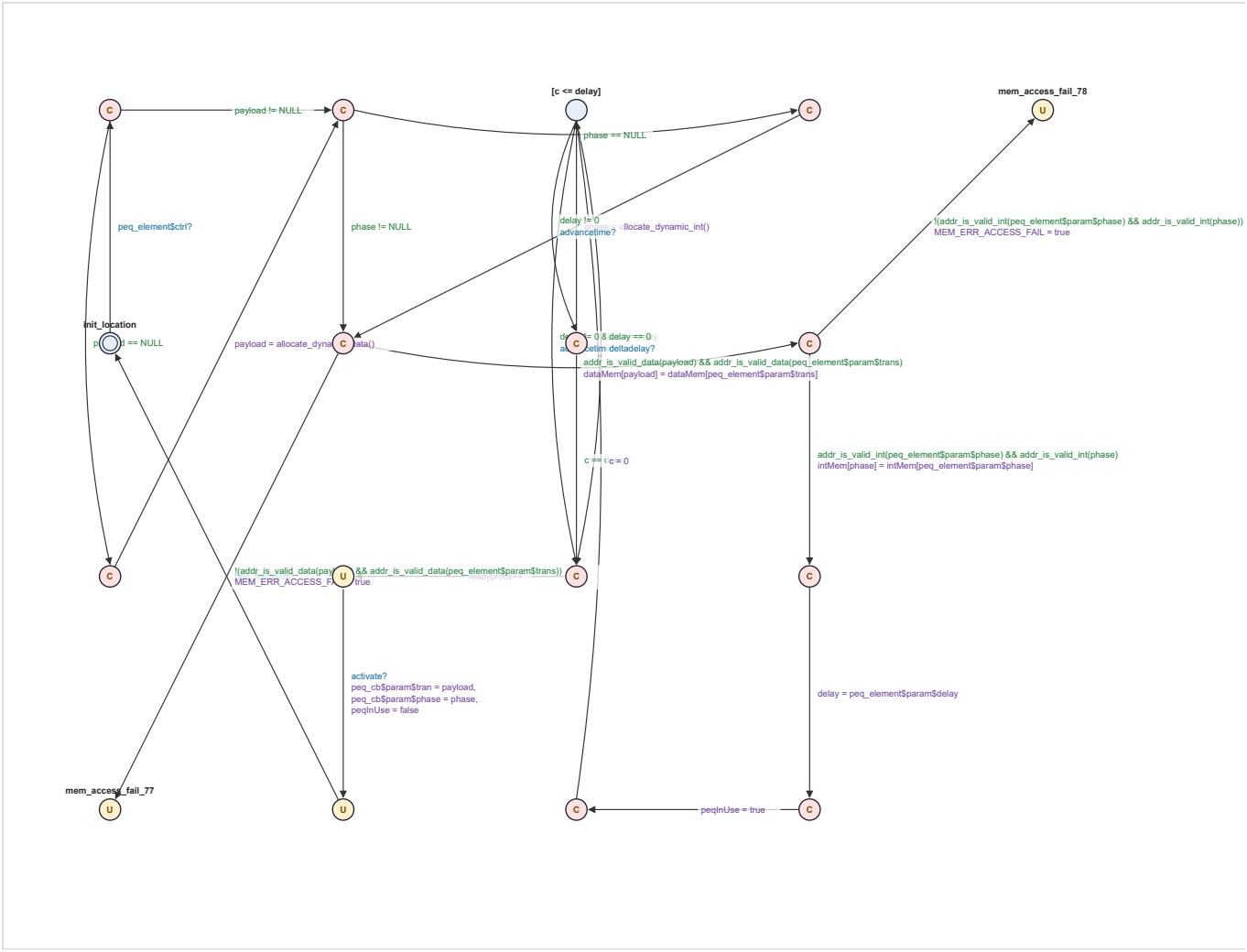


Fig. 2. Template `peq_element$data` from our model, drawn by `tools/make_figures.py` directly from the XML (source-line comments omitted). It corresponds to the PEQ notification automaton of [1, Fig. 4]; the extra branches check pointer validity and set the memory-error flags. U = urgent, C = committed.

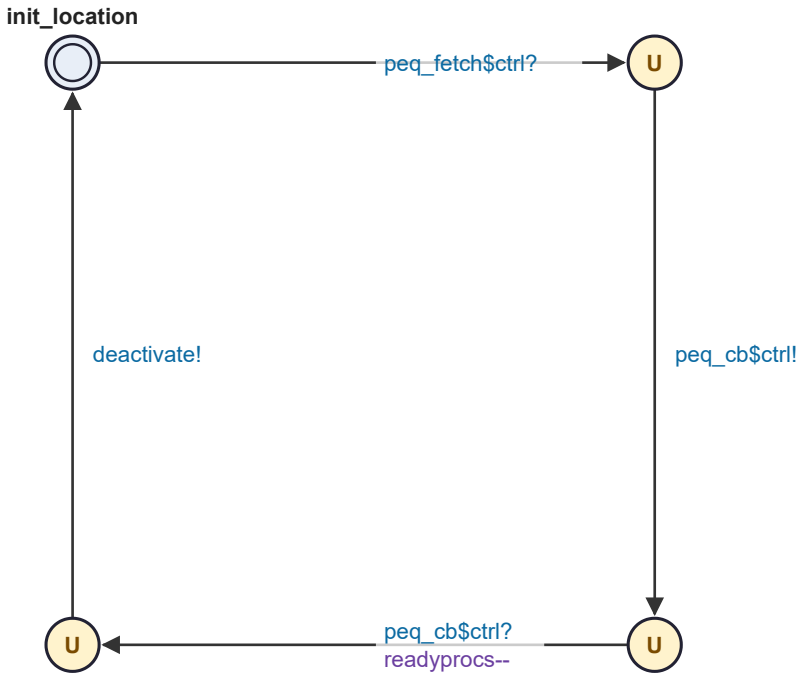


Fig. 3. Template `peq_fetch$data`, the callback invocation automaton (cf. [1, Fig. 5]).

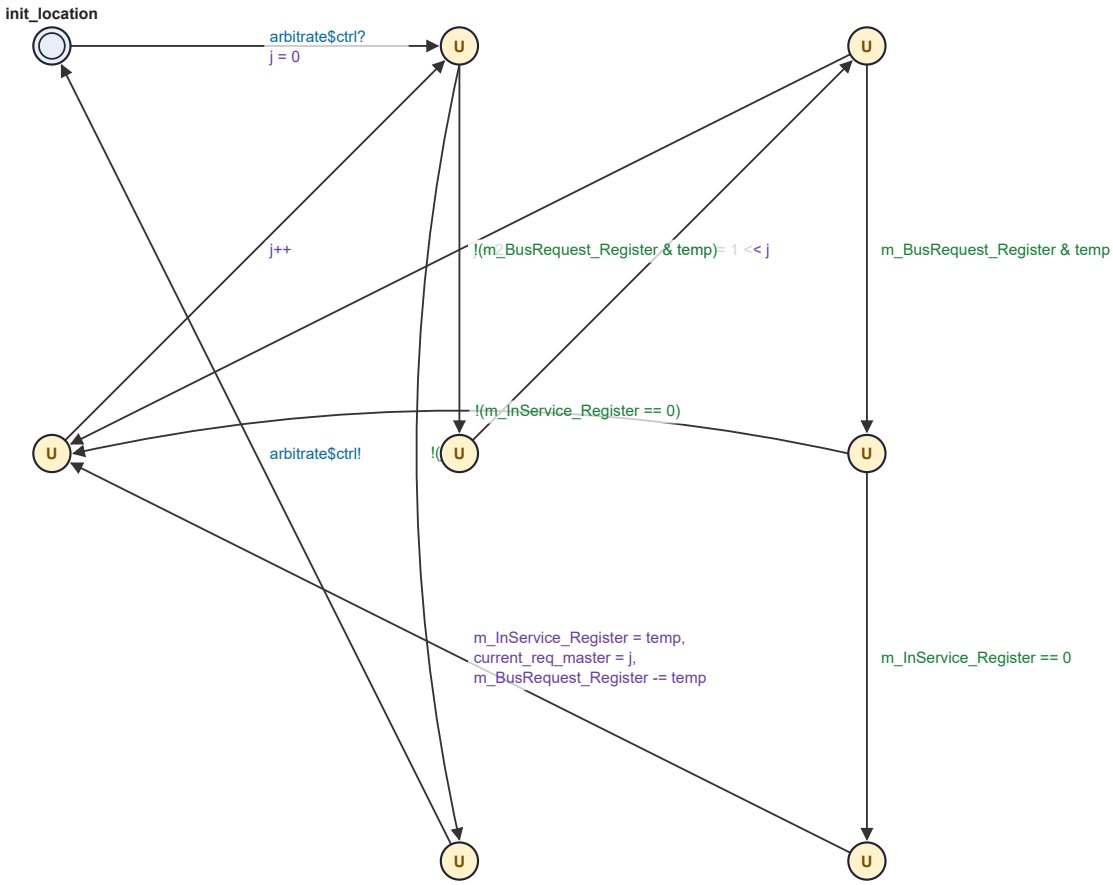


Fig. 4. Template `ahb_simple_bus$arbitrate`. It still loops over two master slots (`j < MASTER_N` in `ahb_simple_bus.cpp`, line 9, with `MASTER_N` = 2 in `defines.h`); in 1M1S only slot 0 is ever requested.

IV. Results

A. Setup

UPPAAL: UPPAAL 5.0.0 (rev. 714BA9DB36F49691), June 2023. Run on 2026-11-10 14:53, Windows 11, Intel64 Family 6 Model 154 Stepping 3, GenuineIntel. All checks used the default search options (`verifyta -u -q`). The paper used a 3.4 GHz Pentium with 4 GB RAM under Linux and averaged 10 runs, so times are not directly comparable. The raw output of every check is in `results/raw/`.

B. Comparison with the paper

Our deadlock-freedom check explored 1,241,387 symbolic states (1,119,525 stored), about 16.7 times the 74,345 states reported in the paper. We do not try to match the paper's count. Possible reasons, none of which we have isolated, are a different UPPAAL version and search options (the paper used a 2011 release; we used the version listed in Section IV-A), our model being reduced by hand from the 2M2S model rather than regenerated, and a PEQ depth of four entries per queue.

The smallest bound that holds is N = 40 time units, which is 2 bus clock periods of 20 time units each. The check with N − 1 = 39 was not satisfied, so 40 is the smallest bound.

TABLE I

Paper Table II (1M1S column) compared with our measurements. Times are CPU time.

Property	Query	Paper time	Paper states	Our CPU time	Our states explored	Our result
Deadlock freedom	`A[] not deadlock`	0:07	74,345	16.38 s	1,241,387 stored 1,119,525	**Satisfied**
Only one master granted	`A[] M1#busgranted + M2#busgranted <= 1`	-	-	*Not applicable: 1M1S has no second master*		
Bus granted to M1	`AHB_M1$m_bus_req_pending --> AHB_M1$m_bus_granted`	0:05	74,345	11.83 s	1,259,923 stored 1,119,525	**Satisfied**
Bus granted to M2	`M2#busrequest --> M2#busgranted`	-	-	*Not applicable: 1M1S has no second master*		
Communication in time	`A[] M1_isCommunicating imply x <= 40`	0:17	74,345	13.02 s	1,241,387 stored 1,119,525	**Satisfied**

The paper reports a single state count per configuration. Our communication-in-time row is measured on the timed model, which has one more clock; see Table II for its other checks.

TABLE II

Additional checks on the 1M1S model and on the timed model.

Check	Query	Model	States explored	CPU time	Result
M1 can be granted the bus (reachability only)	`E<> AHB_M1$m_bus_granted`	amba_AHB_1M1S.uppaal.xml	501	0.01 s	**Satisfied**
Arbiter never grants the unused master-2 slot	`A[] AHB_Bus$m_InService_Register != 2`	amba_AHB_1M1S.uppaal.xml	1,241,387	12.78 s	**Satisfied**
Arbiter grant register only 0 or 1	`A[] AHB_Bus$m_InService_Register <= 1`	amba_AHB_1M1S.uppaal.xml	1,241,387	17.16 s	**Satisfied**
No invalid memory access	`A[] not MEM_ERR_ACCESS_FAIL`	amba_AHB_1M1S.uppaal.xml	1,241,387	16.55 s	**Satisfied**
No double free	`A[] not MEM_ERR_DOUBLE_FREE`	amba_AHB_1M1S.uppaal.xml	1,241,387	36.83 s	**Satisfied**
No out-of-memory	`A[] not MEM_ERR_NOT_ENOUGH_MEM`	amba_AHB_1M1S.uppaal.xml	1,241,387	42.38 s	**Satisfied**
Timed model: deadlock freedom (instrumentation sanity check)	`A[] not deadlock`	amba_AHB_1M1S_timed.uppaal.xml	1,241,387	1:04.6	**Satisfied**
Timed model: communication starts	`E<> M1_isCommunicating`	amba_AHB_1M1S_timed.uppaal.xml	501	0.02 s	**Satisfied**

C. Bug study

With the reset in place, "every started transfer completes" is satisfied. With the reset removed, deadlock freedom is satisfied and "every started transfer completes" is NOT satisfied. So in our model the missing reset shows up as a liveness failure (a transfer that never finishes) rather than as a UPPAAL deadlock, most likely because the clock and scheduler keep running, so the system as a whole is never stuck.

The paper describes the bug in terms of a transaction that is split into several transfers. Our model has no AMBA SPLIT response, so this study only shows what removing the reset does to the bursts that the traffic generator issues; it does not reproduce the paper's exact scenario.

TABLE III

Effect of removing the `m_txn_count` reset.

Check	Query	Model	States explored	CPU time	Result
With reset: every started transfer completes	`M1_isCommunicating --> !M1_isCommunicating`	amba_AHB_1M1S_timed.uppaal.xml	1,799,151	1:13.4	**Satisfied**
Without reset: deadlock freedom	`A[] not deadlock`	amba_AHB_1M1S_timed_noreset.uppaal.xml	57,806	0.69 s	**Satisfied**
Without reset: every started transfer completes	`M1_isCommunicating --> !M1_isCommunicating`	amba_AHB_1M1S_timed_noreset.uppaal.xml	41,403	0.42 s	**Not satisfied**

TABLE IV
Toolchain check on a small SystemC FIFO example (`test_fifo.uppaal.xml`).

Check	Query	Model	States explored	CPU time	Result
FIFO: deadlock freedom	`A[] not deadlock`	test_fifo.uppaal.xml	11,163	0.05 s	**Satisfied**
FIFO: no invalid memory access	`A[] not MEM_ERR_ACCESS_FAIL`	test_fifo.uppaal.xml	11,163	0.02 s	**Satisfied**
FIFO: no memory corruption	`A[] not MEM_ERR_CORRUPTED_MEM`	test_fifo.uppaal.xml	11,163	0.02 s	**Satisfied**

V. Discussion and Limitations

- Our state counts and times differ from the paper's. We report both and have not tried to tune the model or the search options to match.
- The 1M1S model was reduced by hand, not regenerated from SystemC. The SystemC sources of the STATE-2.1 `examples/amba_AHB` design (bus, memory slave, master adapter, traffic generator and `defines.h`) are included in `systemc_source/`. `defines.h` describes the original 2M2S design (`MASTER_N` = 2, `SLAVE_N` = 2); only the traffic generator's `number_of_slaves` was reduced to 1.
- The communication-in-time bound depends on where we start and stop measuring. We start at each grant, as [1] does, and stop when `b_transport` finishes the transaction. Because `x` restarts at every grant, N bounds the time from the most recent grant to the end of the transaction, not the length of the whole transaction. Measuring from the first grant of a transaction would need a second clock and is left for future work.
- UPPAAL 4.1.26-1, which we used first, stopped running because its start-up check failed (it printed an HTTP 301 page). All results here come from the version listed in Section IV-A.

VI. Conclusion

We repeated the 1M1S part of the AMBA AHB experiment of [1] with UPPAAL. For the three properties of Table II that apply to 1M1S, deadlock freedom is satisfied, bus grant to M1 is satisfied, and communication in time holds with N = 40. The bug study shows how the missing counter reset appears in our model. Extending the work to 2M2S would need the full model and far more memory, which is why we limited this assignment to 1M1S.

References

1. M. Pockrandt, P. Herber, and S. Glesner, “Model checking a SystemC/TLM design of the AMBA AHB protocol,” in *Proc. 9th IEEE Symp. Embedded Systems for Real-Time Multimedia (ESTIMedia)*, Taipei, Taiwan, 2011.
2. P. Herber, M. Pockrandt, and S. Glesner, “Transforming SystemC transaction level models into UPPAAL timed automata,” in *Proc. 9th ACM/IEEE Int. Conf. Formal Methods and Models for Codesign (MEMOCODE)*, 2011, pp. 161–170.
3. P. Herber, J. Fellmuth, and S. Glesner, “Model checking SystemC designs using timed automata,” in *Proc. Int. Conf. Hardware/Software Codesign and System Synthesis (CODES+ISSS)*, 2008, pp. 131–136.
4. G. Behrmann, A. David, and K. G. Larsen, “A tutorial on UPPAAL,” in *Formal Methods for the Design of Real-Time Systems*, LNCS 3185, Springer, 2004, pp. 200–236.
5. ARM Ltd., *AMBA Specification (Rev 2.0)*, ARM IHI 0011A, 1999.
6. *IEEE Standard for Standard SystemC Language Reference Manual*, IEEE Std 1666-2011.
7. R. Alur and D. L. Dill, “A theory of timed automata,” *Theoretical Computer Science*, vol. 126, pp. 183–235, 1994.